Neural Networks 201 (2026) 108949

Contents lists available at ScienceDirect

Neural Networks

journal homepage: www.elsevier.com/locate/neunet

Full Length Article

Dynamic expandable framework for incremental anomaly detection

Yuxuan Tan [a,b,1], Hongxia Gao [a,b,c,d,1,*], Tongtong Liu [a,b], Xiaoqin Wen [b]

[a] School of Cyber Science and Engineering, Xi'an Jiaotong University, Xi'an, 710049, China
[b] School of Automation Science and Engineering, South China University of Technology, Guangzhou, 510641, China
[c] Shenzhen Loop Area Institute, Shenzhen, 518000, China
[d] Pazhou Lab, Guangzhou, 510335, China

ARTICLE INFO

Keywords:
Anomaly detection
Incremental learning

ABSTRACT

Incremental anomaly detection (IAD) has gained significant importance due to the evolving nature of product classes in real-world industrial environments. However, existing IAD methods typically rely on a shared model parameter space, which is prone to catastrophic forgetting when new classes are introduced. To fundamentally address this issue, this paper proposes the Dynamic Expandable Framework (DEF), a novel architecture designed to achieve explicit inter-class decoupling and parameter isolation. The methodology integrates three key components: 1) History-Weighted Feature Selection (HWFS), which selects representative anomaly-sensitive channels to mitigate feature conflicts; 2) a Class-Specific Mixture of Experts (CS-MoE) layer, which assigns dedicated parameters to each class to prevent knowledge overwriting; and 3) the Dynamic Contrastive Routing Network (DCRN), which enables accurate expert selection during inference. Furthermore, t-SNE visualization results qualitatively validate the effectiveness of the proposed decoupling mechanism, demonstrating that DEF significantly mitigates inter-class interference within the decoder layers. Extensive experiments on MVTec-AD and VisA benchmarks show that DEF achieves state-of-the-art (SOTA) performance. Specifically, in the MVTec "3-3 with 4 steps" setting, DEF improves Average Accuracy (ACC) by 4.8% and reduces the Forgetting Measure (FM) by 2.15 compared to current leading approaches. Our approach also maintains superior computational efficiency, requiring only 3.52G FLOPs and 4400MB of training memory, offering an optimal balance for real-world industrial deployment.

1. Introduction

Anomaly detection (AD) presents a core challenge across multiple data modalities, including sensor streams, time-series measurements, and multimodal industrial signals. Its objective is to identify instances that deviate significantly from established normal patterns within data distributions (Bono et al., 2023; Zamanzadeh Darban et al., 2024). In recent years, visual anomaly detection has achieved remarkable success across diverse application domains, ranging from agricultural disease identification to sophisticated industrial quality control (Ali et al., 2025; Jebur et al., 2025). Owing to the inherently ambiguous definition of anomalies and the practical scarcity of labeled anomalous samples, unsupervised anomaly detection (UAD) frameworks leveraging exclusively normal data have garnered substantial research interest. Within the specialized domain of computer vision, UAD methods specifically target the identification of anomalous visual patterns in image data (Defard et al., 2021; Lei et al., 2023; Liu et al., 2023; You et al., 2022a; Zavrtanik et al., 2022). However, the practical deployment of these methods in real-world industrial settings faces significant engineering hurdles.

Most existing approaches typically follow a "one-model-per-class" paradigm (Deng & Li, 2022; Roth et al., 2022; Yao et al., 2023; Zavrtanik et al., 2021), where a separate model is trained for each specific product or component. While effective for static scenarios, this paradigm incurs prohibitive memory overhead and maintenance costs when scaled to hundreds of product class on resource-constrained edge devices. To mitigate resource redundancy, unified multi-class anomaly detection methods (Lu et al., 2023; Meng et al., 2024; You et al., 2022a) have been proposed to model multiple classes within a single network. Nevertheless, these unified approaches generally assume a static training set. In the specific context of industrial defect detection, inspection scenarios typically evolve over time. The initial training phase is often unable to cover the full spectrum of object classes; instead, new objects are acquired gradually in accordance with manufacturing schedules and production adjustments. When a new product class emerges, unified methods typically require retraining on the entire dataset (old and new classes) to prevent performance degradation. This process is computationally inefficient and heavily reliant on storage for historical data. Therefore, there is a compelling need

* Corresponding author.
E-mail address: cathy.hxgao@foxmail.com, hxgao@xjtu.edu.cn (H. Gao).
[1] These authors contributed equally to this paper.

https://doi.org/10.1016/j.neunet.2026.108949
Received 14 June 2025; Received in revised form 6 February 2026; Accepted 4 April 2026
Available online 5 April 2026
0893-6080/

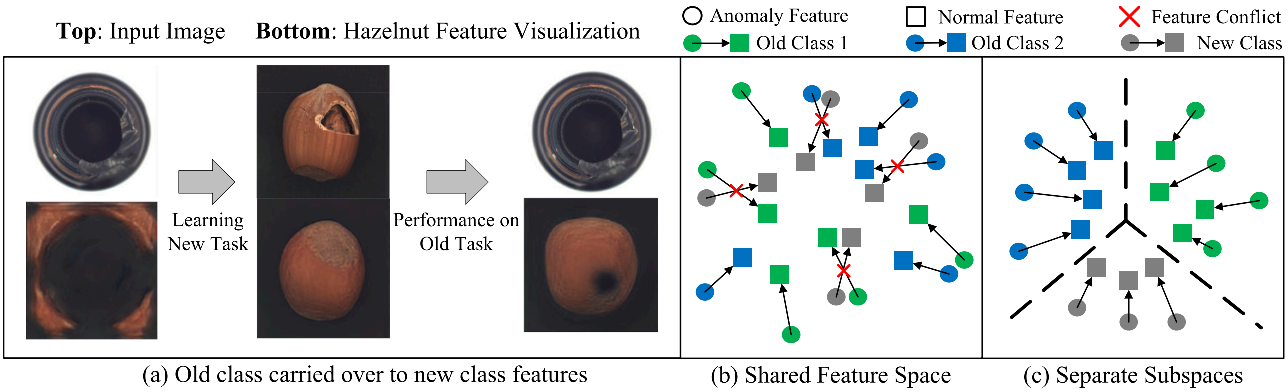

Fig. 1. (a) After learning the new task, the model incorporates features from the new class when reconstructing samples from the old class, leading to failure in anomaly detection. (b) Existing methods share a common feature space, which causes interference between new and old classes. (c) By partitioning separate subspaces and isolating them from each other, the model alleviates catastrophic forgetting.

for incremental anomaly detection that facilitates true streaming updates, eliminating the necessity to retain or re-compute historical data. This paradigm effectively aligns the lifecycle of the model with the dynamic lifecycle of industrial products, allowing the system to adaptively learn new patterns while preserving the memory of previously learned classes.

However, achieving such adaptability poses a fundamental challenge to existing architectures. Specifically, feature reconstruction-based methods often suffer from catastrophic forgetting in incremental learning scenarios (Bugarin et al., 2024; McCloskey & Cohen, 1989; Shin et al., 2017). As illustrated in Fig. 1(a), before the model learns the new class hazelnut, an input from the bottle class cannot reconstruct features associated with hazelnut. After training on the new task, the model successfully reconstructs hazelnut features. However, when reconstructing features for the old class bottle, it also erroneously reconstructs hazelnut features. This phenomenon is common and arises from the use of a shared feature space to learn the normal distributions of all classes in a unified model, as shown in Fig. 1(b). While this approach enables effective utilization of all normal samples during training, it becomes problematic in incremental settings where data from previous tasks is no longer available. In such cases, the model tends to overfit to features of the new task, and the entanglement of features across different classes evolves into conflicts between old and new tasks. As a result, features of old tasks may be reconstructed as those of the new tasks, making anomaly detection ineffective. This has motivated efforts to explore strategies for preserving previously learned knowledge. In recent studies, one line of work (Jin et al., 2024; Liu et al., 2024a; Pang & Li, 2025) builds a memory bank to retain a small amount of data from previous tasks, aiming to mitigate the forgetting of old knowledge. Another line of work (Li et al., 2025; Tang et al., 2024) prevents interference with the parameter space of previously learned knowledge by manipulating gradients during the incremental stage. However, performing incremental learning within a shared model parameter space still inevitably suffers from the risk of catastrophic forgetting.

To fundamentally address these issues, our goal is to decouple different classes and achieve parameter isolation between them. In this paper, we investigate the underlying causes of inter-class coupling in unified models. To alleviate feature conflicts, as illustrated in Fig. 1(c), we explicitly isolate old and new classes by mapping them into separate subspaces. Furthermore, to prevent interference in the parameter space across classes, we assign a dedicated set of model parameters to each class-specific mapping. Specifically, we propose a Dynamic Expandable Framework (DEF) for incremental anomaly detection. It consists of three key components. 1) History-Weighted Feature Selection (HWFS): Before training begins, HWFS selects the most representative anomaly-sensitive channels for the reconstruction task, aiming to alleviate potential channel conflicts across different classes. 2) Class-Specific Mixture of Experts Layer (CS-MoE): By leveraging class-specific experts, this module explicitly achieves inter-class decoupling and enforces parameter isolation between different classes. Additionally, we introduce the Dynamic Contrastive Routing Network (DCRN), which performs class-specific expert routing during the inference stage. 3) Incremental Strategy: During the incremental stage, we expand class-specific experts to learn the normal distribution of new tasks. To prevent overwriting old knowledge, most model parameters are frozen. We conduct experiments on the MVTec-AD and VisA datasets, as well as across both datasets in a cross-dataset setting. The results demonstrate that our method achieves state-of-the-art (SOTA) performance on incremental anomaly detection tasks on both MVTec-AD and cross-dataset scenarios. Meanwhile, our approach maintains extremely low FLOPs and memory overhead, indicating an optimal balance between performance and computational efficiency.

In summary, our contributions can be summarized as follows:

- We investigate the causes of inter-class coupling in unified models, and based on this insight, design a Dynamic Expandable Framework (DEF) for incremental anomaly detection.
- In the incremental learning setting, we generalize Anomaly-aware Features Selection (AFS) to a novel History-Weighted Feature Selection (HWFS) mechanism to mitigate feature conflicts between previously seen and newly introduced classes. We also introduce a Class-Specific Mixture-of-Experts Layer (CS-MoE) and a Dynamic Contrastive Routing Network (DCRN) to enable explicit parameter isolation across tasks. Moreover, we devise an incremental learning strategy that effectively retains knowledge from past tasks.
- Based on our proposed DEF, we conduct extensive experiments. Compared to existing methods, our approach not only achieves SOTA performance but also demonstrates excellent computational efficiency.

2. Related work

This section reviews the most significant prior research in anomaly detection, organized into two subsections. The first subsection discusses conventional unsupervised anomaly detection methods that operate under static data assumptions, while the second subsection covers incremental anomaly detection approaches designed for dynamic scenarios with evolving class distributions.

2.1. Anomaly detection

Due to the difficulty of collecting anomalous samples, most existing studies focus on unsupervised anomaly detection methods that require only normal samples for training. These methods can be broadly categorized into three groups.

1) Embedding-based methods: These approaches detect anomalies by constructing decision boundaries in a high-dimensional space. One-class classification (OCC) methods, such as PatchSVDD (Yi & Yoon, 2020), distinguish normal from anomalous features by enclosing the normal features within a hypersphere. Distribution-map methods model the distribution of normal samples using normalized flow-based mappings to a multivariate Gaussian, and identify anomalies based on deviations from this distribution, as seen in PyramidFlow (Lei et al., 2023), FastFlow (Yu et al., 2021) and FRAnomaly (Milković et al., 2024). Memory bank methods, such as PatchCore (Roth et al., 2022), PaDiM (Defard et al., 2021) and CPR (Li et al., 2024) store key features of normal samples in a memory bank and detect anomalies by computing distances between test features and the memory. Teacher-student architectures rely on a pretrained teacher network and distill knowledge into a student network to learn normal sample patterns. Anomalies are detected by comparing discrepancies between the teacher and student outputs, as employed in methods such as RD4AD (Deng & Li, 2022), AST (Rudolph et al., 2023), URD (Liu et al., 2025), SK-RD4AD (Park et al., 2025), and Skip-TS (Liu et al., 2024b).

2) Reconstruction-based methods: These are based on the assumption that reconstruction networks can accurately reconstruct normal samples but fail to reconstruct anomalous ones. Depending on the reconstruction target, they are further divided into image-level and feature-level reconstruction methods. Image-level reconstruction directly restores input samples to normal RGB images using various architectures: DRAEM (Zavrtanik et al., 2021) employs an autoencoder; Anoseg (Song et al., 2021) uses a generative adversarial network (GAN); DiffusionAD (Zhang et al., 2025) and GLAD (Yao et al., 2024) utilize diffusion models; and AnoViT (Lee & Kang, 2022) adopts a Transformer for reconstruction. Feature-level reconstruction methods first extract features using a pretrained network and then reconstruct those features, as in ADTR (You et al., 2022b) and AMI-Net (Luo et al., 2024). Recently, several studies have extended feature-level reconstruction to multi-class anomaly detection by learning the normal distribution of multiple classes using a unified model. UniAD (You et al., 2022a) addresses the identical shortcut problem and introduces the first Transformer-based unified model. HVQ-Tran (Lu et al., 2023) incorporates vector quantization, and MoEAD (Meng et al., 2024) employs a mixture of experts (MoE) to achieve parameter efficiency. Although these methods have achieved promising results, they do not consider class shifts in dynamic industrial scenarios. When new classes need to be introduced, continuing training on the existing model leads to catastrophic forgetting, necessitating full retraining with access to data from previous tasks.

3) synthesis-based methods: In the absence of genuine anomalous samples, some studies adopt strategies that synthesize pseudo-anomalies by introducing artificial perturbations to construct anomalous data. For instance, CutPaste (Li et al., 2021) simulates anomalies by randomly cropping image patches and pasting them onto other locations within the same image. AnatPaste (Sato et al., 2023) improves upon this by first segmenting semantically meaningful foreground regions and then applying Gaussian blur along the pasting boundaries to produce more natural-looking synthetic anomalies. Beyond operations in the original RGB pixel space, alternative approaches construct pseudo-anomalies directly in feature space. For example, DSR (Zavrtanik et al., 2022) generates feature representations with anomalous semantics by replacing the quantized feature vectors of normal images with other vectors from the codebook. SuperSimpleNet (Rolih et al., 2024), on the other hand, injects Gaussian noise into feature maps to generate perturbed, anomaly-like features. Although these methods rely on artificially crafted anomaly patterns, they effectively enhance the model's ability to discriminate anomalies in scenarios where real anomalous data are unavailable.

2.2. Incremental anomaly detection

A few recent studies have begun to explore anomaly detection under incremental learning scenarios, enabling models to retain anomaly detection capabilities on previously learned classes while learning new ones, thereby offering greater flexibility. Based on their implementation strategies, these methods can be broadly categorized into replay-based and optimization-based approaches, as summarized in Table 1. Replay-based methods aim to mitigate forgetting by storing key features from previous tasks. For example, DNE (Li et al., 2022) stores statistical information of normal sample features but only provides image-level detection. UCAD (Liu et al., 2024a) introduces a key-knowledge-prompt memory architecture for incremental anomaly segmentation. ONER (Jin et al., 2024) performs experience replay using decomposed prompts and semantic prototypes, while CFRDC (Pang & Li, 2025) effectively captures anomaly-identification-related knowledge from limited exemplars of old classes via context-aware feature reconstruction. Optimization-based methods, on the other hand, explicitly manipulate the model's gradient update process during the incremental stage. IUF (Tang et al., 2024) adjusts channel weights by injecting class information and projecting onto the semantic space. CDAD (Li et al., 2025) mitigates interference with the old feature space by introducing gradient projection constraints into diffusion models. Replay-based methods require additional memory to store samples from previous tasks, while optimization-based methods learn both old and new tasks within a shared parameter space, which inevitably leads to catastrophic forgetting. These limitations motivate our work. In contrast to existing paradigms that rely on data retention or gradient-level interventions, we take a fundamentally different approach: by introducing a model architecture-based solution, we explicitly construct class-specific parameter spaces for both old and new classes. This structural separation enables better knowledge retention and significantly alleviates catastrophic forgetting in incremental anomaly detection.

3. Problem setting

We divide the dataset into N mutually disjoint subsets $D = \{D^1, D^2, \dots, D^N\}$, where each subset corresponds to an independent task and contains one or more classes. Let D^t_{train} and D^t_{test} denote the training and test sets of the t-th subset, respectively. The entire training process is divided into N sequential phases. At the t-th phase, only the training subset D^t_{train} is used for model training. After each phase is completed, the model proceeds to the next phase and continues training on D^{t+1}_{train}, where class labels are available for supervision. Once all training phases are finished, the model is evaluated on each test set D^t_{test}, $t \in [1, N]$, without access to class labels. This evaluation aims to assess the stability and plasticity of the model.

4. Method

4.1. Overview

We propose a Dynamic Expandable Framework (DEF), as illustrated in Fig. 2. The complete training process consists of two stages. In the first stage, the History-Weighted Feature Selection (HWFS, see Section 4.2) module is employed to select the optimal channel subset C_{opt} for each class. In the second stage, DEF is trained based on the selected optimal channel subset C_{opt}. DEF consists of three components: first, features are extracted using a frozen pre-trained CNN backbone; second, a Transformer-based reconstruction network with a Class-Specific Mixture-of-Experts Layer (CS-MoE, see Section 4.3) and a Dynamic Contrastive Routing Network (DCRN, see Section 4.4) is employed; finally, feature comparison is performed to localize and detect anomalies.

In the feature extraction stage, we utilize a frozen pre-trained CNN backbone, EfficientNet-B4 (Tan & Le, 2019). Given an input image $I_n \in \mathbb{R}^{C \times H \times W}$, it is passed through the backbone network $\Phi(\cdot)$. Since feature maps at different scales provide complementary information from various perspectives, we extract multi-scale feature maps from multiple

Table 1
Incremental Anomaly Detection Summary.

Paradigm	Method	Highlight	Limitation
Replay-based	DNE Li et al. (2022)	Stores statistical information of normal features	Requires significant auxiliary memory for historical data storage
	UCAD Liu et al. (2024a)	Key-knowledge-prompt memory architecture	
	ONER Jin et al. (2024)	Experience replay via decomposed prompts and semantic prototypes	
	CFRDC Pang and Li (2025)	Context-aware reconstruction with sparse feature storage	
Optimization-based	IUF Tang et al. (2024)	Semantic compression loss and gradient projection strategy	Inevitably suffers from forgetting due to shared parameter space interference
	CDAD Li et al. (2025)	Efficient gradient projection via iterative SVD (iSVD)	

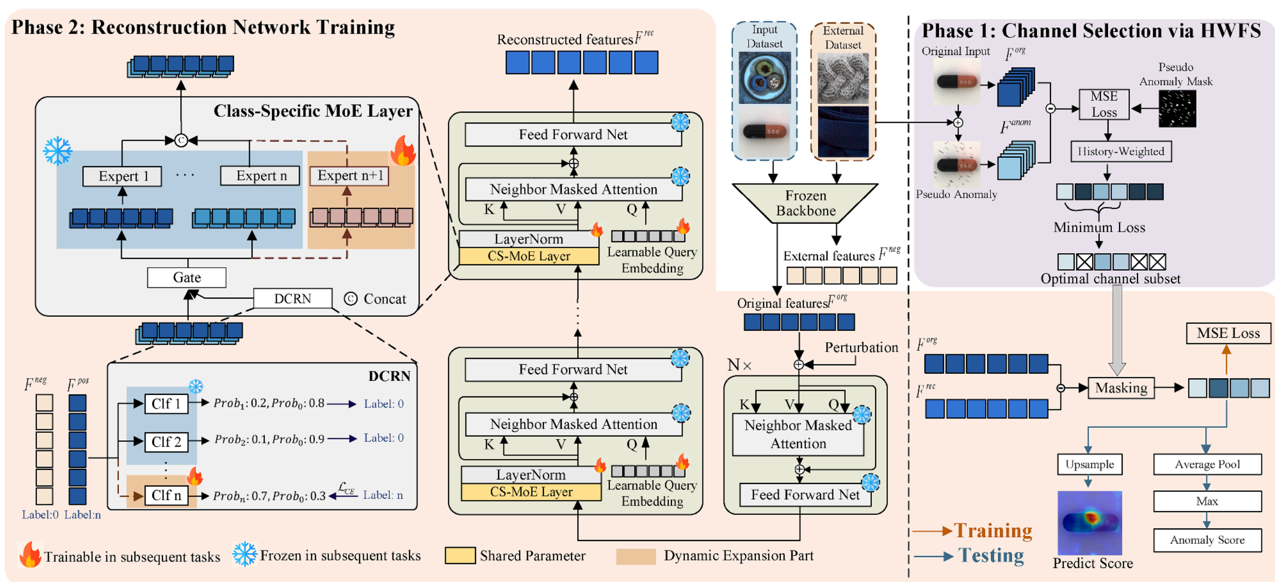

Fig. 2. Overview of the DEF framework featuring a two-stage training process. Phase 1 (Right) utilizes History-Weighted Feature Selection (HWFS) to identify the Optimal Channel Subset by analyzing feature sensitivity to pseudo-anomalies. Phase 2 (Left) trains the reconstruction network incorporating a Class-Specific MoE Layer and DCRN for explicit inter-class decoupling. The two phases are linked via a Masking operation, where the Optimal Channel Subset from Phase 1 guides the MSE Loss computation in Phase 2.

blocks of the network. These feature maps are resized to a uniform spatial resolution and concatenated along the channel dimension to obtain the fused multi-scale feature representation $\Phi(I_n) \in \mathbb{R}^{B\times C\times H\times W}$, where B is the batch size, and C, H, and W denote the number of channels, height, and width of the feature maps, respectively. The fused features are then reshaped into a token sequence, denoted as $F^{\text{org}} \in \mathbb{R}^{N\times B\times C}$, where B is the batch size, N is the total number of feature tokens, and C is the channel dimension. The token is then projected into a smaller dimension C' using a linear layer $Proj_{in}(\cdot)$ and fed into the Transformer for reconstruction.

The reconstruction network consists of an encoder and a decoder, each built by stacking N layers. The encoder processes F^{org} to extract latent features, which are then reconstructed into $F^{\text{rec}} \in \mathbb{R}^{N\times B\times C'}$ by the decoder. The C' dimension of the reconstructed token is then restored to the original C dimension by a linear layer $Proj_{out}(\cdot)$. The entire process is formulated as follows:

$$F^{\text{enc}} = \text{Encoder}(\text{Proj}_{\text{in}}(F^{\text{org}})), \tag{1}$$

$$F^{\text{rec}} = \text{Decoder}(\text{Proj}_{\text{out}}(F^{\text{enc}})). \tag{2}$$

Finally, we compute the anomaly score map s by measuring the difference between the reconstructed tokens F^{rec} and the original tokens F^{org} over the optimal channel subset C_{opt}, followed by bilinear interpolation to up-sample the result to the input image size:

$$s = Upsample\left(\left\|F_c^{\text{rec}} - F_c^{\text{org}}\right\|_2\right) \quad c \in C_{\text{opt}}. \tag{3}$$

Subsequently, we apply average pooling to the anomaly score map s and take the maximum value as the image-level anomaly score.

4.2. History-Weighted feature selection

Due to the high redundancy of features extracted by pre-trained networks and the variability of the optimal channel subset across different datasets and classes, the potential for feature conflicts between new and old tasks is further exacerbated. Inspired by RealNet (Zhang et al., 2024), we extend Anomaly-aware Features Selection (AFS) to class-incremental scenarios and propose an improved strategy, History-Weighted Feature Selection (HWFS). Specifically, we introduce a channel historical importance memory mechanism, which dynamically adjusts the feature selection weights for the current task by recording the performance of the channel in past task.

AFS Loss. Given the j-th normal image I_j^n, perlin noise is sampled and binarized to create a pseudo-anomaly mask M_j. An external texture image I_j^e is randomly selected from a DTD dataset (Cimpoi et al., 2014), and element-wise multiplied by M_j to extract the anomalous region. This region is then superimposed onto I_j^n, generating a pseudo-anomalous image I_j^a:

$$I_j^a = I_j^n \odot (1 - M_j) + I_j^e \odot M_j. \tag{4}$$

Feature extraction is performed on both the normal image I_j^n and the pseudo-anomalous image I_j^a, yielding fused feature representations $\Phi(I_j^n)$ and $\Phi(I_j^a)$. For the i-th channel in the fused feature representation, we compute the squared difference between the normal feature map and the pseudo-anomalous feature map, $[\Phi_i(I_j^n) - \Phi_i(I_j^a)]^2$, to evaluate the capability of that channel in reflecting anomalies. Then, leveraging the pseudo-anomaly mask M_j, we compute the mean squared error, which defines the AFS loss for channel i:

$$\mathcal{L}_{AFS}(\Phi_i) = \frac{1}{J}\sum_{j=1}^{J}\left\|\left[\Phi_i\left(I_j^n\right) - \Phi_i\left(I_j^a\right)\right]^2 - M_j\right\|_2^2. \tag{5}$$

History-weighted AFS loss To mitigate inter-task channel conflicts in incremental learning scenarios and avoid excessive selection of the same channels, we maintain a dynamic weight S_i for each feature channel to reweight $\mathcal{L}_{AFS}$, resulting in $\mathcal{L}_{HWFS}$:

$$\mathcal{L}_{HWFS}(\Phi_i) = F(\mathcal{L}_{AFS}(\Phi_i)) + S_i^{N-1}. \tag{6}$$

Here, the superscript $N-1$ in S_i^{N-1} denotes the $(N-1)$th task, and the function $F(\cdot)$ scales the input to the range of 0 to 1. For the first task, we initialize S_i to 0. In each subsequent task, we use $\mathcal{L}_{AFS}$ to reflect the importance of each channel and update S_i accordingly. For tasks involving multiple classes, we take the average of $\mathcal{L}_{AFS}$ before updating S_i^N:

$$S_i^N = 1 - F\left(\mathcal{L}_{HWFS}(\Phi_i)\right). \tag{7}$$

After that, we select the top K channels with the smallest $\mathcal{L}_{HWFS}$ among all the channels for each class, and save the channel index as the optimal channel subset C_{opt} for that class:

$$C_{\text{opt}} = \arg\min_{C'} \sum_{i\in C'} \mathcal{L}_{HWFS}(\Phi_i), \quad \text{s.t.} \quad |C'| = K. \tag{8}$$

This feature selection process is performed for each class before training. During subsequent training and testing, the reconstruction error is calculated exclusively on the optimal channel subset of each class.

4.3. Class-Specific mixture of experts layer for decoder layer

First, we revisit the attention mechanism (Vaswani et al., 2017). Given a query matrix $Q \in \mathbb{R}^{N\times C}$, a key matrix $K \in \mathbb{R}^{N\times C}$, and a value matrix $V \in \mathbb{R}^{N\times C}$, the attention weights are computed based on the similarity between Q and K, which are then used to perform a weighted aggregation of V:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{9}$$

where $\sqrt{d_k}$ is the scaling factor. As in previous unified approaches (You et al., 2022a), a single learnable Query is employed in the decoder layer, while the Key and Value are derived from the output of the encoder. In this reconstruction process, the role of the Query is more akin to "attention guidance" rather than directly encoding class information, with the Key and Value taking on a dominant role. However, in incremental learning scenarios, the Key and Value, which carry specific class information, share the same parameter space. This leads to interclass interference between new and old tasks, resulting in catastrophic forgetting.

Class-specific mixture of experts To alleviate feature entanglement across different classes, we propose a Class-Specific Mixture of Experts (CS-MoE) Layer, where each expert is responsible for processing features from a specific class only. This approach mitigates the entanglement of features from different classes in the "Key" and "Value" representations through explicit parameter isolation. At the same time, it constructs independent parameter spaces for each class, preventing the overwriting of learned representations for existing classes when incorporating new ones. Specifically, we define a variable number of experts in the set $E = \{E_1, E_2, \dots, E_N\}$, where each expert E_i is implemented as a dedicated linear transformation layer (fully-connected layer) that processes features exclusively for one specific class. For the i-th sample in the batch feature F, which belongs to class j, denoted as $F_{i,j} \in \mathbb{R}^{N\times C}$, it is fed into the expert corresponding to that class, yielding the output $Z_i \in \mathbb{R}^{N\times C}$. The tokens output by different experts are concatenated to obtain $Z \in \mathbb{R}^{N\times B\times C}$. This process can be expressed as:

$$Z_j = E_j(F_{i,j}), \tag{10}$$

$$Z = Concat(Z_1, Z_2, \dots, Z_B). \tag{11}$$

The feature distributions produced by different experts exhibit variations. We apply LayerNorm to Z, ensuring that features across different classes maintain consistent scales:

$$Z' = LayerNorm(Z). \tag{12}$$

The resulting output serves as Key and Value, and we compute attention with a learnable Query $Q_{\text{learnable}}$ using Neighbor Masked Attention (NMA), inspired by UniAD (You et al., 2022a), to prevent the generation of identical shortcuts. Finally, the process passes through a feed-forward network (FFN), with specific details as follows:

$$Q = Q_{\text{learnable}} W_Q, \tag{13}$$

$$K = Z' W_K, \tag{14}$$

$$V = Z' W_V, \tag{15}$$

$$F^{out} = FFN(NMA(Q, K, V) + Z'). \tag{16}$$

We implement incremental learning by introducing new experts (see Section 4.5). To prevent a rapid increase in the number of parameters, each class-specific expert shares its parameters across different decoder layers.

Expert selection mechanism During the training phase, expert selection is straightforward and deterministic, samples from class j are routed exclusively to expert E_j based on their class labels. This supervised routing ensures that each expert learns to reconstruct features only for its designated class.

During inference when class labels are unavailable, the Dynamic Contrastive Routing Network (DCRN, see Section 4.4) takes over the expert selection process. As detailed in Section 4.4, DCRN functions as a dynamic gating mechanism that predicts the most appropriate expert for each input sample by evaluating confidence scores from multiple class-specific classification heads. This allows our framework to maintain class-specific parameter isolation even in the absence of class labels during testing.

4.4. Dynamic contrastive routing network

We extended the concept of dynamic incremental experts and designed a Dynamic Contrastive Routing Network (DCRN). Specifically, we define a variable-sized set of classification heads: $Clf = \{Clf_1, Clf_2, \dots, Clf_K\}$. Where each classification head Clf_k corresponds to a specific target class. The target classes serve as positive samples, while in addition to other classes from the same training batch as negative samples, we also introduce images from external DTD dataset (Cimpoi et al., 2014) as negative samples. Using $\Phi(\cdot)$ to extract features from both positive and negative samples, we obtain $F^{pos} \in \mathbb{R}^{N\times B\times C}$ and $F^{neg} \in \mathbb{R}^{N\times B\times C}$ respectively. During training, for each sample $F_i^{pos} \in \mathbb{R}^{N\times C}$ in the batch, we randomly replace it with $F_i^{neg} \in \mathbb{R}^{N\times C}$ according to a predefined probability. Subsequently, average pooling is applied to generate image representation vectors $f_i \in \mathbb{R}^C$, which are then fed into

Table 2
Quantitative performance comparison of different methods on the MVTec-AD dataset, expressed as image-level results/pixel-level results. The best and second-best results are highlighted in **bold** and underlined, respectively.

Method	Year	14 – 1 with 1 Step		10 – 5 with 1 Step		3 – 3 with 4 Steps		10 – 1 with 5 Steps	
		ACC(↑)	FM(↓)	ACC(↑)	FM(↓)	ACC(↑)	FM(↓)	ACC(↑)	FM(↓)
UniAD	NeurIPS'22	85.7 / 89.6	18.3 / 13.3	86.7 / 91.5	14.9 / 10.6	81.3 / 88.7	7.4 / 10.6	76.6 / 82.3	21.1 / 17.3
UniAD + EWC	-	92.8 / 95.4	4.1 / 1.9	90.5 / 93.6	7.3 / 4.2	79.6 / 89.0	9.5 / 10.1	89.6 / 93.8	5.4 / 3.6
UniAD + SI	-	85.7 / 89.5	18.4 / 13.4	84.1 / 88.3	20.2 / 17.0	81.9 / 88.5	7.0 / 10.8	77.2 / 81.6	20.2 / 18.2
UniAD + MAS	-	85.8 / 89.6	18.1 / 13.3	86.8 / 91.0	14.9 / 11.6	81.5 / 89.0	7.2 / 10.2	77.9 / 82.0	19.5 / 17.7
UniAD + LVT	-	80.4 / 86.0	29.1 / 20.6	87.1 / 90.6	14.1 / 12.3	80.4 / 88.6	8.6 / 10.6	78.2 / 88.3	19.1 / 16.1
MoEAD	ECCV'24	74.2 / 79.5	23.5 / 17.9	72.2 / 81.3	29.7 / 19.2	70.1 / 79.1	16.0 / 11.7	69.9 / 77.8	9.8 / 5.5
MambaAD	NeurIPS'24	74.0 / 87.6	25.2 / 10.5	71.6 / 85.5	31.5 / 14.8	65.2 / 73.8	19.2 / 15.9	61.8 / 72.3	14.6 / 8.8
DiAD	AAAI'24	93.5 / 96.5	1.7 / 0.1	91.9 / 93.9	3.2 / 3.9	80.5 / 89.0	11.8 / 7.2	83.3 / 92.7	12.2 / 3.6
Dinomaly	CVPR'25	95.7 / 96.2	4.2 / 2.2	<u>96.0</u> / <u>96.0</u>	4.4 / 3.1	77.1 / 83.7	20.1 / 13.6	79.3 / 81.4	13.0 / 9.0
ViTAD	CVIU'25	95.6 / 96.5	2.5 / 1.1	89.7 / 93.3	10.0 / 5.5	69.1 / 79.1	21.5 / 17.0	87.2 / 93.2	3.6 / 1.1
IUF	ECCV'24	96.0 / 96.3	1.0 / 0.6	92.2 / 94.4	9.3 / 6.3	84.2 / 91.1	10.0 / 8.4	94.2 / 95.1	3.2 / 1.0
CDAD	CVPR'25	<u>96.4</u> / <u>96.6</u>	**-0.57** / **0.04**	94.2 / 95.3	<u>2.05</u> / <u>2.40</u>	<u>89.1</u> / <u>92.2</u>	<u>3.69</u> / <u>4.65</u>	<u>94.9</u> / <u>95.7</u>	<u>1.01</u> / <u>0.74</u>
DEF (Ours)	-	**97.2** / **96.8**	<u>0.08</u> / <u>0.00</u>	**97.2** / **97.0**	**0.14** / **0.04**	**93.9** / **96.1**	**1.54** / **0.32**	**96.4** / **96.7**	**0.28** / **0.04**

their corresponding classification heads. We employ the cross-entropy loss function:

$$\mathcal{L}_{ce} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \sigma(Clf_k(f_i)) + (1-y_i)\log(1-\sigma(Clf_k(f_i)))\right], \tag{17}$$

where $y_i = 1$ for positive samples and $y_i = 0$ for negative samples, and $\sigma(\cdot)$ denotes the sigmoid activation function. During inference, given an input image feature vector t_i, the predicted class is determined by selecting the classification head with the highest confidence score, resulting in the following prediction:

$$y_i = \arg\max_n \left[Clf_1(f_i), \dots, Clf_K(f_i)\right]. \tag{18}$$

4.5. Incremental strategy

Incremental protocol During the incremental stage, thanks to the use of class-specific experts, we only need to add new experts to the expert set E to learn the distribution of newly introduced classes. Simultaneously, new classification heads are added to the classification head set Clf to ensure that tokens are routed to the correct expert during inference. To prevent forgetting and better retain previously learned knowledge, we train only the newly added experts, the LayerNorm modules at each decoder layer, the learnable Query $Q_{\text{learnable}}$, and the new classification heads Clf_N, while freezing all other parameters. However, there is still a risk of knowledge forgetting, as the parameters of LayerNorm and Query after parameter updates may not align with the old task data. Therefore, we apply Exponential Moving Average (EMA) for parameter updates:

$$\theta_{t+1} = \theta_t - (1-\beta)\nabla\theta_t, \tag{19}$$

where β is the weighting factor, θ_t represents the LayerNorm parameters at time step t, and $\nabla\theta_t$ is the current gradient update. Compared to the case without EMA updates, this effectively applies decay to the gradients of the task-shared parameters. We aim to first train the class-specific experts during the incremental phase and then fine-tune the task-shared parameters. Regarding the number of newly introduced parameters when learning each new class, it amounts to only 65,808, which is approximately 0.9% of the total model parameters (approximately 6.66M before learning any classes).

Adaptability to distribution shifts While our standard incremental protocol freezes the experts of previously learned classes to maximize stability, the explicit parameter isolation in the CS-MoE layer offers inherent flexibility for handling intra-class distribution shifts (i.e., concept drift). In real-world scenarios where the definition of "normal" for a specific class evolves (e.g., lighting changes or material updates), our framework allows for selectively unfreezing and fine-tuning the corresponding expert E_i without affecting the parameters of other classes. This targeted adaptation capability ensures that the model can track dynamic changes in established tasks while avoiding the interference typically seen in shared-parameter architectures.

4.6. Loss function

Our total loss function consists of equally weighted reconstruction loss $\mathcal{L}_{rec}$ and DCRN's cross-entropy loss $\mathcal{L}_{ce}$, as both are crucial to their respective tasks. The reconstruction loss $\mathcal{L}_{rec}$ is computed as the Mean Squared Error (MSE) on the optimal channel subset C_{opt}:

$$\mathcal{L}_{\text{rec}} = \frac{1}{|C_{\text{opt}}|}\sum_{c\in C_{\text{opt}}}\left\|F_c^{\text{rec}} - F_c^{\text{org}}\right\|^2 \quad c \in C_{\text{opt}}, \tag{20}$$

$$\mathcal{L} = \mathcal{L}_{rec} + \mathcal{L}_{ce}. \tag{21}$$

5. Experiments

5.1. Experimental setup

Dataset MVTec-AD (Bergmann et al., 2019) and VisA (Zou et al., 2022) are commonly used benchmark datasets in the field of anomaly detection. MVTec-AD covers 15 industrial object classes with a total of 5354 images, while VisA includes 12 different object classes with 10,821 images. In our experiments, we chose these two datasets.

Protocol To ensure a fair and rigorous comparison with existing state-of-the-art methods, we strictly adhere to the established incremental learning protocols (Li et al., 2025; Tang et al., 2024). We define our experimental settings using the notation "$\mathcal{B} - \mathcal{I}$ with K step(s)", where $\mathcal{B}$ represents the number of object classes in the initial (base) task, $\mathcal{I}$ is the class increment size per step, and K is the total number of incremental steps (tasks) following the base task. Based on this established protocol, the specific splits chosen for the MVTec-AD dataset (15 classes total) are: 14 – 1 with 1 step, 10 – 5 with 1 step, 3 – 3 with 4 steps, and 10 – 1 with 5 steps. Similarly, for the VisA dataset (12 classes total), the adopted settings include 11 – 1 with 1 step, 8 – 4 with 1 step, and the multi-step setting 8 – 1 with 4 steps.

Metrics We follow the established evaluation protocols in incremental anomaly detection (Li et al., 2022, 2025; Tang et al., 2024) to assess the model's stability and plasticity. The fundamental metric for anomaly detection performance is the Area Under the Receiver Operating Characteristic (AUROC). To measure performance in a continual learning setting, we construct a performance matrix A, where $A_{N,i}$ specifically denotes the test AUROC (image-level or pixel-level) of task i after the

Table 3
Quantitative performance comparison of different methods on the VisA dataset, expressed as image-level results/pixel-level results. The best and second-best results are highlighted in **bold** and underlined, respectively.

Method	Year	11 – 1 with 1 Step		8 – 4 with 1 Step		8 – 1 with 4 Steps	
		ACC(↑)	FM(↓)	ACC(↑)	FM(↓)	ACC(↑)	FM(↓)
UniAD	NeurIPS'22	75.0 / 92.1	22.4 / 11.4	78.1 / 94.0	14.7 / 8.4	72.2 / 90.8	16.6 / 9.2
UniAD + EWC	-	78.7 / 95.4	14.9 / 4.8	80.5 / 95.4	10.0 / 5.3	72.3 / 92.0	16.5 / 7.3
UniAD + SI	-	78.1 / 92.0	16.9 / 11.5	80.8 / 93.9	9.2 / 8.3	69.8 / 88.5	19.8 / 12.0
UniAD + MAS	-	75.4 / 91.8	21.5 / 11.9	78.4 / 94.0	14.1 / 8.4	72.1 / 90.6	16.7 / 9.4
UniAD + LVT	-	77.5 / 92.3	17.3 / 10.9	78.8 / 94.1	13.4 / 8.1	70.8 / 91.4	18.3 / 8.4
MoEAD	ECCV'24	63.6 / 87.1	25.5 / 11.2	66.1 / 88.2	25.4 / 11.9	63.7 / 86.3	10.6 / 2.7
MambaAD	NeurIPS'24	65.8 / 89.8	29.4 / 9.0	72.2 / 89.9	24.3 / 9.8	66.4 / 91.5	5.2 / 4.6
DiAD	AAAI'24	70.2 / 91.1	5.6 / 2.2	67.4 / 91.7	7.7 / 1.6	65.0 / 84.9	11.2 / 8.8
Dinomaly	CVPR'25	**93.6** / 95.9	5.6 / 3.0	79.1 / 93.7	23.5 / 6.0	72.9 / 84.7	13.9 / 11.3
ViTAD	CVIU'25	85.0 / 96.7	5.3 / 1.5	69.7 / 94.5	24.0 / 4.2	70.6 / 91.5	7.5 / 4.6
IUF	ECCV'24	87.3 / 97.6	2.4 / 1.8	80.1 / 95.4	9.8 / 6.8	79.8 / 95.0	9.8 / 6.8
CDAD	CVPR'25	88.3 / 97.2	**-1.28** / **-0.06**	**85.3** / 97.1	1.39 / 0.29	**83.4** / 95.8	3.79 / 1.49
DEF(Ours)	-	88.5 / **98.0**	0.24 / 0.05	85.1 / **97.7**	**0.11** / **0.02**	82.8 / **97.0**	**0.54** / **0.11**

model has completed training on task N. Based on this AUROC matrix, we report the Average Accuracy (ACC) and Forgetting Measure (FM). ACC represents the average AUROC across all learned tasks after the final training phase, reflecting the model's overall detection capability. FM quantifies the decline in AUROC for previous tasks as new tasks are introduced, reflecting the severity of catastrophic forgetting. These are defined as follows:

$$\mathrm{ACC} = \frac{1}{N}\sum_{i=1}^{N} A_{N,i}, \tag{22}$$

$$FM = \frac{1}{N-1}\sum_{i=1}^{N-1} \max_{b\in\{1,\cdots,N-1\}} \left(A_{b,i} - A_{N,i}\right), \tag{23}$$

where N is the total number of tasks.

Implementation details All training images are resized to $224 \times 224 \times 3$. We utilize a pretrained EfficientNet-B4 (Tan & Le, 2019) model to extract feature maps from Stage-1 to Stage-4. These feature maps are resized and concatenated along the channel dimension, resulting in a unified feature map of size $14 \times 14 \times 272$. This feature map is then projected to 256 dimensions and fed into a Transformer network consisting of 4 encoder layers and 4 decoder layers for feature reconstruction. The reconstructed features are subsequently projected back to 272 dimensions for anomaly localization. We employ sinusoidal positional encoding and set the exponential moving average (EMA) decay rate β to 0.999 in the incremental strategy. The model is optimized using the AdamW optimizer (Loshchilov & Hutter, 2017) with a learning rate of 1×10^{-4} and a weight decay of 0.0001. Training is performed with a batch size of 32 on a single NVIDIA GeForce RTX 4090 GPU. The initial task is trained for 500 epochs to ensure sufficient learning of fundamental feature representations. Each subsequent incremental task is trained for 100 epochs, as the parameter search space is substantially reduced during incremental learning, and limiting the training epochs helps prevent both overfitting to new tasks and forgetting of previously acquired knowledge.

5.2. Performance comparison

Quantitative evaluation We conducted a comprehensive comparison of image-level detection and pixel-level localization methods on the MVTec-AD and VisA datasets. These methods include multi-class anomaly detection methods such as UniAD (You et al., 2022a), MambaAD (He et al., 2024a), DiAD (He et al., 2024b), Dinomaly (Guo et al., 2025), ViTAD (Zhang et al., 2023), traditional incremental learning strategies like EWC (Kirkpatrick et al., 2017), SI (Zenke et al., 2017), MAS (Aljundi et al., 2018), LVT (Wang et al., 2022), the mixture of experts method MoEAD (Meng et al., 2024) (similar to our method), and methods specifically designed for incremental anomaly detection such as IUF (Tang et al., 2024) and CDAD (Li et al., 2025). For the MVTec-AD dataset, the results in Table 2 show that our method (DEF) outperforms all other methods across all tasks, except for the FM metric in the "14 – 1 with 1 Step" setting, where we achieved the second-best result. Specifically, in the "3 – 3 with 4 Steps" setting, where more classes need to be learned in subsequent incremental tasks, our method still demonstrates strong resistance to catastrophic forgetting, with performance improvements of 4.8/3.9 and FM reduced by 2.15/4.33 compared to the best existing methods. Notably, while multi-class SOTAs like Dinomaly and DiAD exhibit impressive static performance, they suffer from significant forgetting under incremental settings. For the VisA dataset, the results in Table 3 show that our method consistently exhibits lower forgetting metrics, performs best in pixel-level ACC, and shows highly competitive results in other metrics as well. Overall, these quantitative results strongly indicate that our method surpasses existing methods in terms of overall performance, not only showing significant advantages in anomaly detection and localization tasks but also demonstrating outstanding resistance to forgetting.

Cross-Dataset Performance. To evaluate the generalization ability of our method across different datasets, we conducted cross-dataset incremental learning experiments. Specifically, we performed cross-training between the MVTec-AD and VisA datasets, where "MVTec-VisA" denotes training on MVTec-AD first, followed by VisA, and vice versa for "VisA-MVTec". As shown in Table 4, DEF outperforms UniAD combined with traditional incremental strategies such as EWC and SI, the latest multi-class anomaly detection methods, and methods specifically designed for incremental anomaly detection in all scenarios. These results highlight DEF's exceptional generalization and robustness in cross-dataset incremental learning tasks.

Visualization Comparison. Fig. 3 presents a comparison of anomaly localization results under the "3 – 3 with 4 Steps" setting on the MVTec-AD dataset. We use a color gradient (ranging from blue to red) to indicate anomaly scores, where blue represents lower anomaly likelihood and red signifies higher anomaly probability. MoEAD suffers from severe forgetting issues, particularly for the cable class in the first task, where it nearly loses its localization capability. In contrast, our method consistently maintains accurate anomaly localization across both new and previously learned tasks, demonstrating its superior ability to retain and adapt knowledge in continual learning scenarios.

5.3. Complexity comparison

In addition to detection capability and resistance to forgetting, computational complexity is a critical factor determining whether a model can be successfully deployed in real-world industrial applications. To evaluate computational efficiency, we compare three key aspects:

Table 4
Quantitative performance comparison of different methods on cross-dataset, expressed as image-level results/pixel-level results. The best and second-best results are highlighted in **bold** and underlined, respectively.

Method	Year	MVTec–VisA		VisA–MVTec	
		ACC (↑)	FM (↓)	ACC (↑)	FM (↓)
UniAD	NeurIPS'22	75.0 / 92.1	22.4 / 11.4	78.1 / 94.0	14.7 / 8.4
UniAD + EWC	-	71.4 / 84.5	24.8 / 15.1	65.1 / 86.7	27.9 / 12.8
UniAD + SI	-	87.6 / 94.9	5.8 / 2.1	79.1 / 93.2	13.2 / 5.6
UniAD + MAS	-	85.4 / 94.5	9.3 / 2.9	82.1 / 95.1	8.5 / 2.6
UniAD + LVT	-	73.0 / 83.9	23.7 / 15.7	69.9 / 87.7	28.7 / 13.7
MoEAD	ECCV'24	63.6 / 87.1	25.5 / 11.2	66.1 / 88.2	25.4 / 11.9
MambaAD	NeurIPS'24	72.5 / 83.8	31.6 / 18.1	72.2 / 88.6	31.3 / 13.0
DiAD	AAAI'24	85.5 / 93.0	10.2 / 4.2	78.1 / 92.2	12.5 / 5.9
Dinomaly	CVPR'25	87.5 / 90.4	15.3 / 10.2	80.2 / 90.7	26.2 / 10.8
ViTAD	CVIU'25	75.3 / 87.2	27.1 / 13.4	74.9 / 89.6	24.4 / 11.5
IUF	ECCV'24	82.4 / 92.5	11.09 / 4.88	74.3 / 89.7	16.74 / 5.54
CDAD	CVPR'25	90.0 / 94.9	4.71 / 1.78	84.2 / 94.1	6.90 / 3.45
DEF (Ours)	-	**93.2 / 96.6**	**1.02 / 0.25**	**86.4 / 95.9**	**3.70 / 1.73**

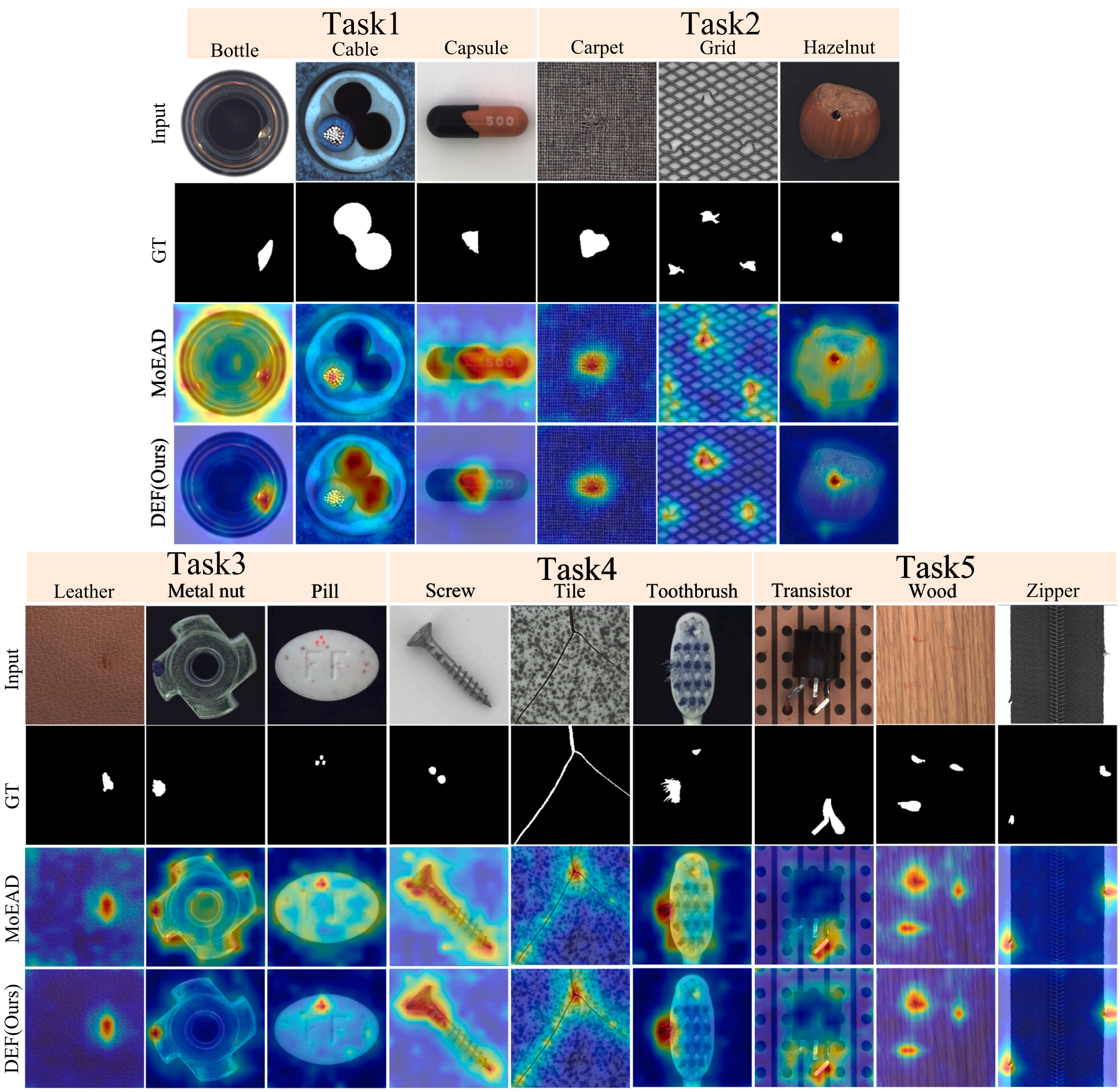

Fig. 3. Comparison of anomaly localization results under the "3 – 3 with 4 Steps" setting of the MVTec.

Table 5
Efficiency comparison of different incremental anomaly detection SoTA methods. Since our model is a dynamically expandable model, the parameters of our model are those at increments up to 15 classes.

Method	FLOPs	Parameters	Training Memory
IUF	183.22G	9.11M	14361MB
CDAD	5499.68G	892M	22102MB
DEF (Ours)	**3.52G**	**7.66M**	**4400MB**

Table 6
Time breakdown for each module of DEF.

Module	Inference Time(ms)	Percentage(%)
Feature Extraction	51.73	58.3%
DCRN Routing	0.20	0.2%
Feature Reconstruction	36.85	41.5%
Total Inference Time	**88.78**	**100%**

FLOPs, trainable parameters, and training memory consumption. All evaluations were conducted on a single RTX 4090 GPU, with a batch size of 4 for training memory measurement. As shown in Table 5, our method outperforms existing incremental anomaly detection approaches across all three metrics, particularly in terms of FLOPs, demonstrating its exceptionally low computational complexity. These results indicate that our method effectively balances detection performance, resistance to forgetting, and computational efficiency, making it well-suited for industrial applications.

We performed a detailed time breakdown of the complete inference process. Table 6 shows the average inference time of each component when using $224 \times 224 \times 3$ input images. The measurements were conducted using PyTorch's CUDA event timers for precise timing, with each configuration running 1715 inference iterations (excluding the first 10 iterations to eliminate cache effects). The results indicate that feature reconstruction and feature extraction constitute the main computational overhead, while the DCRN is relatively lightweight, accounting for only 0.2% of the total inference time.

5.4. Ablation studies

5.4.1. Overall ablation

We conducted a series of ablation experiments to evaluate the effectiveness of our proposed HWFS, CS-MoE and DCRN module. The following ablation experiments are based on the “3 – 3 with 4 Steps” setting of the MVTec-AD dataset.

Effectiveness of CS-MoE. First, we incorporate CS-MoE into the baseline model, where the routing network is implemented as a multi-class softmax classifier. In the first task, the ACC improves from 92.2/97.7 to 97.1/98.2 (+4.9/+0.5), and the model exhibits slower forgetting over the first three tasks. However, after learning all tasks, it still suffers from severe forgetting, which is attributed to the inability of the routing network to correctly select the class-specific expert. Additionally, Fig. 4(a) presents the t-SNE (Van der Maaten & Hinton, 2008) visualization of the Key and Value across different decoder layers, revealing that the class coupling phenomenon is especially pronounced when the decoder is not added to the CS-MoE layer. Although the encoder achieves feature separation between classes, as the decoder deepens, the feature distributions of different classes transition from separation in shallow layers to overlap in deeper layers. As shown in Fig. 4(b), with the addition of the CS-MoE layer, the explicit isolation of features between different classes is achieved thanks to the class-specific experts, which significantly mitigates the inter-class coupling phenomenon in the Key and Value layer spaces of the different decoder layers.

To provide a deeper insight into this decoupling mechanism and address the interpretability of the experts, we further conducted an expert-switching experiment on the MVTec dataset. The goal was to validate whether class-specific experts indeed learn features unique to their assigned classes. In this experiment, test samples were deliberately routed to non-corresponding experts during inference. As shown in Table 8, when samples are processed by incorrect experts, performance significantly decreases across all experimental settings. This substantial performance degradation confirms that each expert indeed learns features highly specific to the normal appearance of its assigned class, supporting our design rationale of explicit parameter isolation between classes.

Effectiveness of DCRN. We analyze the routing accuracy of tokens to class-specific experts. As shown in Table 9, the original multi-class softmax classifier exhibits severe misrouting under incremental learning scenarios, particularly on the MVTec-AD “3 – 3 with 4 Steps” setting and the Visa “8 – 1 with 4 Steps” and “8 – 4 with 1 Step” setting. In contrast, our proposed DCRN maintains consistently high routing accuracy, achieving 100% correct routing in nearly half of the tasks, with the remaining accuracies all above 90%. Furthermore, replacing the softmax-based routing network with DCRN on top of CS-MoE leads to a significant performance boost. Not only is forgetting effectively mitigated, but the image-level ACC across all tasks remains above 90%. The final ACC improves from 71.4/83.3 to 90.8/96.1 (+19.4/+12.8), and the FM drops significantly from 18.12/9.62 to 2.86/0.44 (-15.26/-9.18). Besides, to investigate the impact of external datasets on DCRN, we replaced the DTD dataset (Cimpoi et al., 2014) with Imagenette2 (a subset of ImageNet (Deng et al., 2009)) as negative samples while keeping all other components unchanged. As shown in “DCRN with Imagenette2” in Table 9, DCRN maintained high routing accuracy and achieved even higher accuracy under certain settings. These results confirm that the effectiveness of DCRN hinges on the presence of external negative samples. The flexibility in selecting external datasets enhances the practicality of this method across various industrial scenarios.

Effectiveness of HWFS. Finally, we incorporate HWFS, which further improves the final task performance. The ACC increases from 90.8/96.1 to 93.9/96.1 (+3.1/+0.0), while the FM decreases from 2.86/0.44 to 1.54/0.32 (-1.32/-0.12). This demonstrates that the introduction of HWFS alleviates channel conflicts between old and new class, thereby further enhancing the model's forgetting resistance.

5.4.2. Analysis of the IS

Effectiveness of IS. We evaluate the effectiveness of the proposed incremental strategy. In Table 10, the 'w/o IS' setting denotes the absence of our incremental strategy, where model parameters are fully updated during the incremental phase. This results in a significant performance drop on the final task: ACC decreases from 93.9/96.1 to 82.2/92.8 (-11.7/-3.3), and FM increases from 1.54/0.32 to 12.11/3.93 (+10.57/+3.61), indicating a higher degree of forgetting. These results demonstrate that our incremental strategy effectively preserves knowledge from previous tasks.

Analysis of EMA Decay Rate. To investigate the impact of the decay rate parameter in EMA, we conducted an ablation study on the MVTec dataset under the “3 – 3 with 4 Steps” setting. As shown in Table 11, the results indicate that the model is insensitive to the EMA decay rate, and applying EMA consistently outperforms the scenario without EMA, achieving a lower forgetting metric (FM) and higher accuracy (ACC), thereby demonstrating the effectiveness of EMA.

5.4.3. Analysis of HWFS

Hyper-parametric Analysis. We investigate the performance of HWFS with different sizes of the optimal channel subset, i.e., different values of K in Eq. 8. As shown in Fig. 5, the subset size has a significant impact on performance. When a small number of channels is selected (e.g. $K = 64$), the model exhibits the worst accuracy and severe forgetting, particularly at the image level. On the other hand, selecting a larger subset leads to lower FM, but at the cost of reduced image-level ACC. In contrast, choosing a moderate number of channels yields the best overall performance. Specifically, setting $K = 128$ achieves the highest

Table 7
An ablation study is performed on the different components of our proposed model.

CS-MoE	DCRN	HWFS	Task1	Task2	Task3	Task4	Task5	FM
✗	✗	✗	92.2/97.7	69.0/84.0	77.5/84.9	73.1/80.1	74.1/75.9	12.94/15.69
✓	✗	✗	97.1/**98.2**	92.3/88.8	85.8/90.5	72.5/85.5	71.4/83.3	18.12/9.62
✓	✓	✗	**97.2/98.2**	95.9/**97.8**	94.4/97.2	92.3/96.6	90.8/96.1	2.86/0.44
✓	✓	✓	97.1/98.1	**97.1/97.8**	**96.2/97.3**	**94.5/96.7**	**93.9/96.1**	**1.54/0.32**

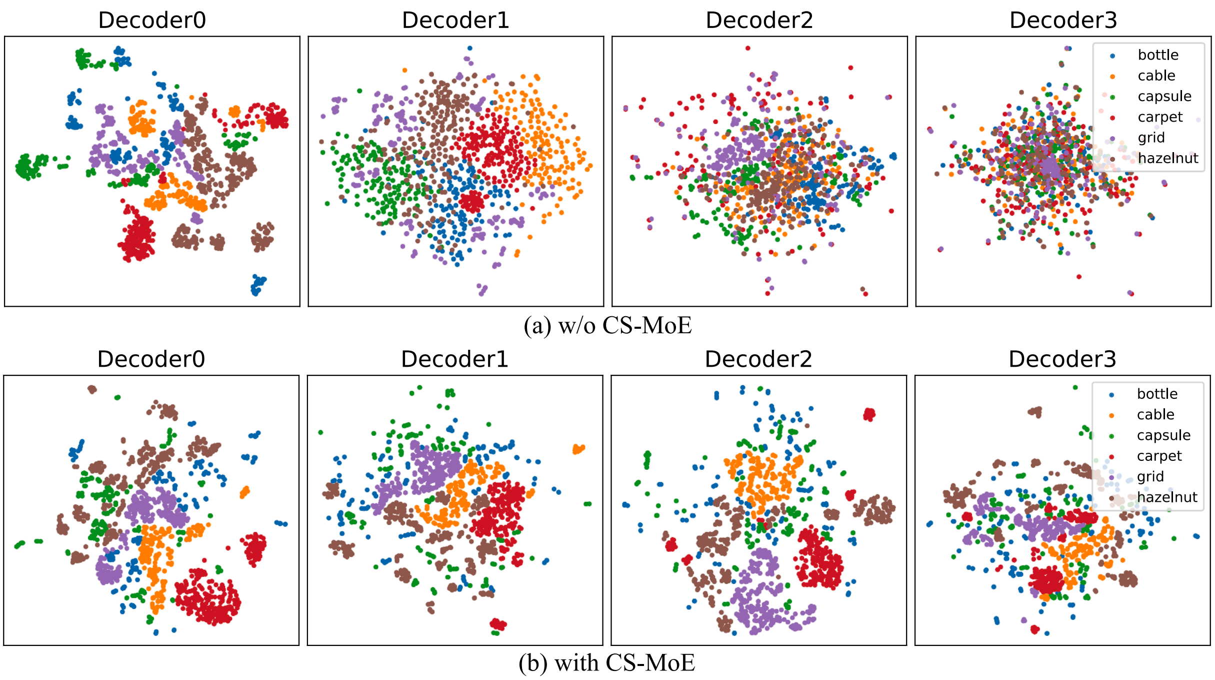

Fig. 4. T-SNE (Van der Maaten & Hinton, 2008) visualization of the Key and Value features from different decoder layers after learning the first task (bottle, cable, capsule) followed by the second task (carpet, grid, hazelnut): (a) When CS-MoE is not added, inter-class coupling becomes severe as the decoder layer deepens; (b) The addition of CS-MoE alleviates inter-class coupling in the decoder layer.

Table 8
Performance comparison when routing test samples to correct vs. incorrect class-specific experts on MVTec-AD. ACC results are expressed at the image level/pixel level.

Settings	Correct Expert Routing	Incorrect Expert Routing
14 – 1 with 1 Step	97.2 / 96.8	82.5 / 87.4
10 – 5 with 1 Step	97.2 / 97.0	73.4 / 78.8
3 – 3 with 4 Steps	93.9 / 96.1	70.2 / 75.1
10 – 1 with 5 Steps	96.4 / 96.7	67.4 / 75.6

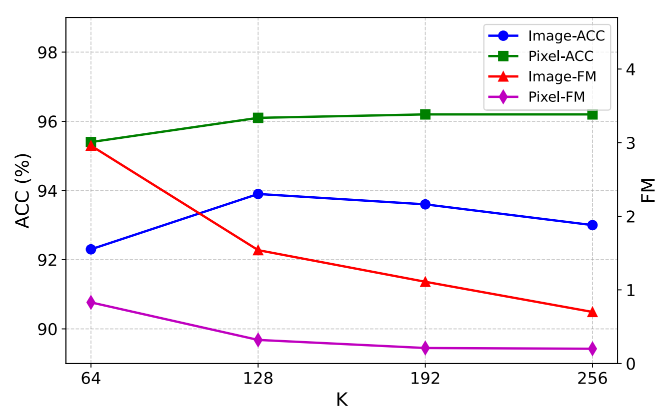

Fig. 5. Variation of ACC and FM for different optimal channel subset sizes.

ACC. These results indicate that selecting a moderately sized channel subset for each class can effectively mitigate channel conflicts in incremental learning scenarios, further validating the effectiveness of HWFS. In practical scenarios, we recommend selecting a moderate value of K to balance detection performance and forgetting, or determining the value of K by reserving a small portion of each class as a validation set.

Visualizations of Channel Weights. Under the “3 – 3 with 4 Steps” experimental setting on the MVTec-AD dataset, we analyzed the channel weight changes when learning the second task for the carpet class. As shown in Fig. 6, we present the original channel weights, the history-weighted channel weights, and their differences (with all channels sorted in descending order of original weights). The “Difference Before and After History-Weighted” represents the result of subtracting the original channel weights from the history-weighted channel weights, where negative values indicate channel deprioritization and positive values indicate weight enhancement. The visualization results demonstrate that the HWFS mechanism dynamically balances the channel importance of new tasks, thereby preventing feature conflicts between old and new tasks. This helps the model maintain high performance during incremental learning.

5.4.4. Robustness analysis of DCRN across similar classes

DCRN is crucial for inference. To evaluate its robustness in distinguishing visually similar classes, we conducted experiments on two carefully selected sets of highly similar class pairs from the MVTec (Bergmann et al., 2019), VisA (Zou et al., 2022), and RealIAD (Wang et al., 2024) datasets (Fig. 7: (a) macaroni1 and macaroni2 from VisA;

Table 9
Class-specific routing accuracy of DCRN versus softmax classifier under different experimental settings, where "DCRN with Imagenette2" uses Imagenette2 as the external dataset.

Dataset	Settings	Multi-class Classifier (%)	DCRN (%)	DCRN with Imagenette2 (%)
MVTec	14 – 1 with 1 Step	100.00	100.00	100.00
	10 – 5 with 1 Step	98.78	100.00	100.00
	3 – 3 with 4 Steps	60.69	96.41	99.53
	10 – 1 with 5 Steps	98.02	99.94	100.00
VisA	11 – 1 with 1 Step	99.93	100.00	100.00
	8 – 4 with 1 Step	78.47	91.60	96.73
	8 – 1 with 4 Steps	55.27	91.67	91.59
MVTec-VisA	-	89.44	100.00	97.63
VisA-MVTec	-	95.76	98.99	98.83

Table 10
Ablation study of incremental strategy (IS).

	Task1	Task2	Task3	Task4	Task5	FM
w/o IS	**97.1/98.1**	91.7/97.2	86.3/95.9	84.7/94.5	82.2/92.8	12.11/3.93
Ours	**97.1/98.1**	**97.1/97.7**	**96.2/97.3**	**94.5/96.7**	**93.9/96.1**	**1.54/0.32**

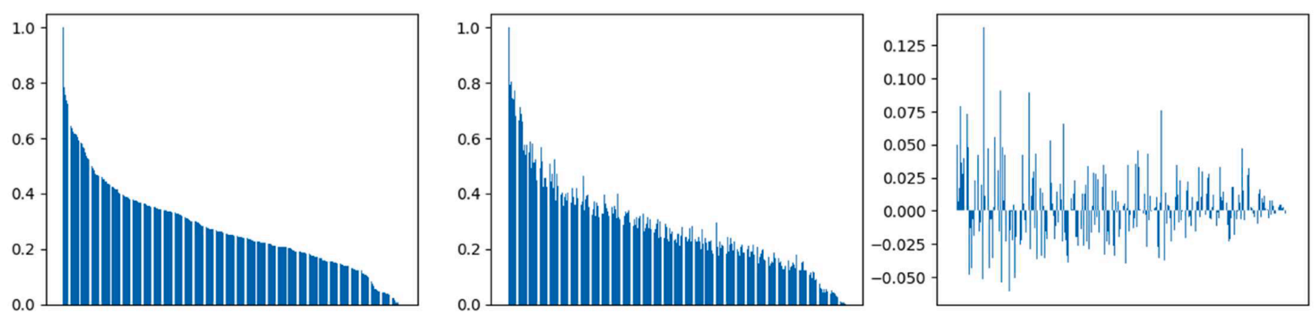
(a) Original Channel Weight (b) Channel Weights with History-Weighted (c) Difference Before and After History-Weighted

Fig. 6. Original channel weights, channel weights with history-weighted, and their differences for the carpet class during learning the second task in the MVTec-AD 3 – 3 with 4 Steps experiment setup. All channels are sorted in descending order by original weight.

Fig. 7. Classes with visual similarity in the MVTec, VisA, and RealIAD datasets.

Table 11
Performance comparison of different EMA decay rate parameters in IS.

EMA Decay	ACC	FM
0.999	**93.9/96.1**	**1.1/0.3**
0.99	93.8/**96.1**	1.0/**0.3**
0.9	**93.9/96.1**	**1.1/0.3**
w/o EMA	92.3/95.6	2.5/0.6

Table 12
Routing accuracy of different similarity classes on DCRN.

Settings	Routing Accuracy
macaroni1 - macaroni2	**100.0**
zipper	**100.0**

and (b) zipper samples from RealIAD (left) and MVTec (right). As shown in Table 12, DCRN achieved 100% accuracy across all pairs of visually similar classes. This indicates that even in scenarios involving visually similar classes, CS-MoE can still receive correctly routed features.

6. Conclusion

In this paper, we introduced the Dynamic Expandable Framework (DEF), a novel architecture specifically designed to address the fundamental challenge of catastrophic forgetting in incremental anomaly detection. To fundamentally resolve the inter-class interference prevalent in models with shared parameter spaces, DEF achieves explicit inter-class decoupling and parameter isolation through the Class-Specific Mixture of Experts (CS-MoE) layer, which assigns dedicated and independent model parameters to each specific class.

The framework incorporates History-Weighted Feature Selection (HWFS), which identifies anomaly-sensitive channels to select the most representative feature subset, thereby effectively mitigating feature conflicts between previously seen and newly introduced classes. Furthermore, to address the challenge where class labels are unavailable during the inference phase, the Dynamic Contrastive Routing Network (DCRN) was proposed as a routing mechanism to enable accurate expert selection, achieving nearly 100% routing accuracy across multiple experimental settings. Extensive experimental evaluations on the MVTec-AD and VisA benchmarks confirm that DEF establishes a new state-of-the-art in incremental learning scenarios. Specifically, in the “3-3 with 4 steps” MVTec-AD setting, DEF improved the Average Accuracy (ACC) by 4.8% and reduced the Forgetting Measure (FM) by 2.15 compared to current leading approaches. The effectiveness of the parameter isolation mechanism is qualitatively supported by t-SNE visualizations, which demonstrate that the CS-MoE layer significantly mitigates inter-class coupling within the decoder layers. Additionally, DEF maintains exceptional computational efficiency, requiring only 3.52G FLOPs and 4400MB of training memory, providing an optimal balance between high-performance detection and resource requirements for real-world industrial deployment.

Limitations & Future Works. Although DEF exhibits excellent performance, this study does not consider class similarity. Additionally, while the parameter count of class-specific experts is relatively small, it becomes a non-negligible burden when the number of new classes increases significantly. Moreover, the current framework relies on image-level routing and processes all feature tokens uniformly, where the interference of redundant background features may constrain detection precision, and the lack of granularity limits capability in complex scenes. Therefore, in future research, we will focus on feature sharing among similar classes. This research is valuable as it reduces the strong dependence on class labels by merging parameter spaces for similar classes, while achieving slower model parameter growth. Furthermore, we plan to investigate adaptive inference strategies, including redundancy-aware pruning and token-wise routing. Specifically, pruning aims to enhance detection robustness by filtering out non-informative artifacts, while token-wise routing leverages CS-MoE’s parameter isolation to handle multi-class images by processing distinct regions independently. These strategies aim to ensure an optimal trade-off between model complexity, versatility, and detection accuracy.

CRediT authorship contribution statement

Yuxuan Tan: Conceptualization, Formal analysis, Methodology, Software, Validation, Writing – review & editing; **Hongxia Gao :** Writing – review & editing, Supervision, Methodology, Funding acquisition, Formal analysis, Conceptualization; **Tongtong Liu:** Writing – review & editing, Visualization, Validation, Methodology; **Xiaoqin Wen:** Writing – review & editing, Visualization, Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

Affiliations a and b contributed equally as the first affiliations for this work. This work was supported in part by the Basic Scientific Research Operating Expenses of Xi’an Jiaotong University under Grants XXJ042025001, the Science and Technology Project of Guangzhou under Grants 202103010003, the Science and Technology Project in key areas of Foshan under Grants 2020001006285, the Xijiang Innovation Team Project under Grants XJCXTD3-2019-04B.

References

Ali, L. R., Jebur, S. A., Jahefer, M. M., Nawar, A. K., & Mahdi, Z. S. (2025). Integrating squeeze-and-excitation network with pretrained CNN models for accurate plant disease detection. *International Journal of Electrical and Computer Engineering Systems*, *16*(8), 621–632.

Aljundi, R., Babiloni, F., Elhoseiny, M., Rohrbach, M., & Tuytelaars, T. (2018). Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European conference on computer vision (ECCV)* (pp. 139–154).

Bergmann, P., Fauser, M., Sattlegger, D., & Steger, C. (2019). Mvtec ad–A comprehensive real-world dataset for unsupervised anomaly detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 9592–9600).

Bono, F. M., Radicioni, L., & Cinquemani, S. (2023). A novel approach for quality control of automated production lines working under highly inconsistent conditions. *Engineering Applications of Artificial Intelligence*, *122*, 106149.

Bugarin, N., Bugaric, J., Barusco, M., Pezze, D. D., & Susto, G. A. (2024). Unveiling the anomalies in an ever-changing world: A benchmark for pixel-level anomaly detection in continual learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 4065–4074).

Cimpoi, M., Maji, S., Kokkinos, I., Mohamed, S., & Vedaldi, A. (2014). Describing textures in the wild. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 3606–3613).

Defard, T., Setkov, A., Loesch, A., & Audigier, R. (2021). Padim: A patch distribution modeling framework for anomaly detection and localization. In *International conference on pattern recognition* (pp. 475–489). Springer.

Deng, H., & Li, X. (2022). Anomaly detection via reverse distillation from one-class embedding. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 9737–9746).

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on computer vision and pattern recognition* (pp. 248–255). Ieee.

Guo, J., Lu, S., Zhang, W., Chen, F., Li, H., & Liao, H. (2025). Dinomaly: The less is more philosophy in multi-class unsupervised anomaly detection. In *Proceedings of the computer vision and pattern recognition conference* (pp. 20405–20415).

He, H., Bai, Y., Zhang, J., He, Q., Chen, H., Gan, Z., Wang, C., Li, X., Tian, G., & Xie, L. (2024a). Mambaad: Exploring state space models for multi-class unsupervised anomaly detection. *Advances in Neural Information Processing Systems*, *37*, 71162–71187.

He, H., Zhang, J., Chen, H., Chen, X., Li, Z., Chen, X., Wang, Y., Wang, C., & Xie, L. (2024b). A diffusion-based framework for multi-class anomaly detection. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 8472–8480). (*vol. 38*).

Jebur, S. A., Mohammed, M. A., Ali, L. R., & Abd, D. H. (2025). Mix-hybrid convolutional neural network framework with explainable artificial intelligence for fig leaves disease detection. *International Journal of Intelligent Engineering & Systems*, *18*(4).

Jin, Y., Zhu, J., Wang, G., Li, S., Zhang, J., Liu, X., Liu, Q., & Wang, Y. (2024). Oner: Online experience replay for incremental anomaly detection. arXiv:2412.03907.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A. et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, *114*(13), 3521–3526.

Lee, Y., & Kang, P. (2022). Anovit: Unsupervised anomaly detection and localization with vision transformer-based encoder-decoder. *IEEE Access*, *10*, 46717–46724.

Lei, J., Hu, X., Wang, Y., & Liu, D. (2023). Pyramidflow: High-resolution defect contrastive localization using pyramid normalizing flow. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 14143–14152).

Li, C.-L., Sohn, K., Yoon, J., & Pfister, T. (2021). Cutpaste: Self-supervised learning for anomaly detection and localization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 9664–9674).

Li, H., Hu, J., Li, B., Chen, H., Zheng, Y., & Shen, C. (2024). Target before shooting: Accurate anomaly detection and localization under one millisecond via cascade patch retrieval. *IEEE Transactions on Image Processing*, .

Li, W., Zhan, J., Wang, J., Xia, B., Gao, B.-B., Liu, J., Wang, C., & Zheng, F. (2022). Towards continual adaptation in industrial anomaly detection. In *Proceedings of the 30th ACM international conference on multimedia* (pp. 2871–2880).

Li, X., Tan, X., Chen, Z., Zhang, Z., Zhang, R., Guo, R., Jiang, G., Chen, Y., Qu, Y., Ma, L. et al. (2025). One-for-more: Continual diffusion model for anomaly detection. arXiv:2502.19848.

Liu, J., Wu, K., Nie, Q., Chen, Y., Gao, B.-B., Liu, Y., Wang, J., Wang, C., & Zheng, F. (2024a). Unsupervised continual anomaly detection with contrastively-learned prompt. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3639–3647). (*vol. 38*).

Liu, M., Jiao, Y., Lu, J., & Chen, H. (2024b). Anomaly detection for medical images using teacher–student model with skip connections and multiscale anomaly consistency. *IEEE Transactions on Instrumentation and Measurement*, *73*, 1–15.

Liu, X., Wang, J., Leng, B., & Zhang, S. (2025). Unlocking the potential of reverse distillation for anomaly detection. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 5640–5648). (*vol. 39*).

Liu, Z., Zhou, Y., Xu, Y., & Wang, Z. (2023). Simplenet: A simple network for image anomaly detection and localization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 20402–20411).

Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization. arXiv:1711.05101.

Lu, R., Wu, Y., Tian, L., Wang, D., Chen, B., Liu, X., & Hu, R. (2023). Hierarchical vector quantized transformer for multi-class unsupervised anomaly detection. *Advances in Neural Information Processing Systems*, *36*, 8487–8500.

Luo, W., Yao, H., Yu, W., & Li, Z. (2024). Ami-net: Adaptive mask inpainting network for industrial anomaly detection and localization. *IEEE Transactions on Automation Science and Engineering*, .

Van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, *9*(11).

McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: the sequential learning problem. In *Psychology of learning and motivation* (pp. 109–165). Elsevier (*vol. 24*).

Meng, S., Meng, W., Zhou, Q., Li, S., Hou, W., & He, S. (2024). MoEAD: A parameter-efficient model for multi-class anomaly detection. In *European conference on computer vision* (pp. 345–361). Springer.

Milković, F., Posilović, L., Medak, D., Subašić, M., Lončarić, S., & Budimir, M. (2024). Franomaly: Flow-based rapid anomaly detection from images. *Applied Intelligence*, *54*(4), 3502–3515.

Pang, J., & Li, C. (2025). Context-aware feature reconstruction for class-incremental anomaly detection and localization. *Neural Networks*, *181*, 106788.

Park, E., Kim, T., Kim, M., Lee, H., & Lee, G.-J. (2025). Sk-rd4ad: Skip-connected reverse distillation for robust one-class anomaly detection. In *Proceedings of the computer vision and pattern recognition conference* (pp. 3945–3953).

Rolih, B., Fučka, M., & Skočaj, D. (2024). Supersimplenet: Unifying unsupervised and supervised learning for fast and reliable surface defect detection. In *International conference on pattern recognition* (pp. 47–65). Springer.

Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., & Gehler, P. (2022). Towards total recall in industrial anomaly detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 14318–14328).

Rudolph, M., Wehrbein, T., Rosenhahn, B., & Wandt, B. (2023). Asymmetric student-teacher networks for industrial anomaly detection. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision* (pp. 2592–2602).

Sato, J., Suzuki, Y., Wataya, T., Nishigaki, D., Kita, K., Yamagata, K., Tomiyama, N., & Kido, S. (2023). Anatomy-aware self-supervised learning for anomaly detection in chest radiographs. *IScience*, *26*(7).

Shin, H., Lee, J. K., Kim, J., & Kim, J. (2017). Continual learning with deep generative replay. *Advances in Neural Information Processing Systems*, *30*.

Song, J., Kong, K., Park, Y.-I., Kim, S.-G., & Kang, S.-J. (2021). Anoseg: Anomaly segmentation network using self-supervised learning. arXiv:2110.03396.

Tan, M., & Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning* (pp. 6105–6114). PMLR.

Tang, J., Lu, H., Xu, X., Wu, R., Hu, S., Zhang, T., Cheng, T. W., Ge, M., Chen, Y.-C., & Tsung, F. (2024). An incremental unified framework for small defect inspection. In *European conference on computer vision* (pp. 307–324). Springer.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, *30*.

Wang, C., Zhu, W., Gao, B.-B., Gan, Z., Zhang, J., Gu, Z., Qian, S., Chen, M., & Ma, L. (2024). Real-iad: A real-world multi-view dataset for benchmarking versatile industrial anomaly detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 22883–22892).

Wang, Z., Liu, L., Duan, Y., Kong, Y., & Tao, D. (2022). Continual learning with lifelong vision transformer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 171–181).

Yao, H., Liu, M., Yin, Z., Yan, Z., Hong, X., & Zuo, W. (2024). Glad: Towards better reconstruction with global and local adaptive diffusion models for unsupervised anomaly detection. In *European conference on computer vision* (pp. 1–17). Springer.

Yao, X., Li, R., Qian, Z., Luo, Y., & Zhang, C. (2023). Focus the discrepancy: Intra-and inter-correlation learning for image anomaly detection. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 6803–6813).

Yi, J., & Yoon, S. (2020). Patch SVDD: Patch-level svdd for anomaly detection and segmentation. In *Proceedings of the Asian conference on computer vision*.

You, Z., Cui, L., Shen, Y., Yang, K., Lu, X., Zheng, Y., & Le, X. (2022a). A unified model for multi-class anomaly detection. *Advances in Neural Information Processing Systems*, *35*, 4571–4584.

You, Z., Yang, K., Luo, W., Cui, L., Zheng, Y., & Le, X. (2022b). Adtr: Anomaly detection transformer with feature reconstruction. In *International conference on neural information processing* (pp. 298–310). Springer.

Yu, J., Zheng, Y., Wang, X., Li, W., Wu, Y., Zhao, R., & Wu, L. (2021). Fastflow: Unsupervised anomaly detection and localization via 2d normalizing flows. arXiv:2111.07677.

Zamanzadeh Darban, Z., Webb, G. I., Pan, S., Aggarwal, C., & Salehi, M. (2024). Deep learning for time series anomaly detection: A survey. *ACM Computing Surveys*, *57*(1), 1–42.

Zavrtanik, V., Kristan, M., & Skočaj, D. (2021). Draem-a discriminatively trained reconstruction embedding for surface anomaly detection. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 8330–8339).

Zavrtanik, V., Kristan, M., & Skočaj, D. (2022). Dsr–a dual subspace re-projection network for surface anomaly detection. In *European conference on computer vision* (pp. 539–554). Springer.

Zenke, F., Poole, B., & Ganguli, S. (2017). Continual learning through synaptic intelligence. In *International conference on machine learning* (pp. 3987–3995). PMLR.

Zhang, H., Wang, Z., Zeng, D., Wu, Z., & Jiang, Y.-G. (2025). DiffusionAD: Norm-guided one-step denoising diffusion for anomaly detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, .

Zhang, J., Chen, X., Wang, Y., Wang, C., Liu, Y., Li, X., Yang, M.-H., & Tao, D. (2023). Exploring plain vit reconstruction for multi-class unsupervised anomaly detection. arXiv:2312.07495.

Zhang, X., Xu, M., & Zhou, X. (2024). Realnet: A feature selection network with realistic synthetic anomaly for anomaly detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 16699–16708).

Zou, Y., Jeong, J., Pemula, L., Zhang, D., & Dabeer, O. (2022). Spot-the-difference self-supervised pre-training for anomaly detection and segmentation. In *European conference on computer vision* (pp. 392–408). Springer.